### Listing 3.19 Defining a Razor section with the `@section` directive

```
@section ThingsToDoWidget{
    <p>Visit Eiffel Tower</p>
}
```

You render the content of a section within the layout page by placing a call to the `RenderSectionAsync` method at the location you want the content to appear. The method has two versions: one that takes the section's name as a string and another that also takes a Boolean indicating whether all content pages are required to define the section (`true`) or optional (`false`). In the next example, only the destination page will provide content for the section, so it is optional for all other pages. Consequently, you will use the overload that enables you to pass `false` to the second parameter:

```
@await RenderSectionAsync("ThingsToDoWidget", false)
```

As it happens, there is already a call to the `RenderSectionAsync` method that refers to the scripts section in the default project layout file, located just before the closing body element:

```
    <script src="~/lib/jquery/dist/jquery.min.js"></script>
    <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
    <script src="~/js/site.js" asp-append-version="true"></script>

    @await RenderSectionAsync("Scripts", required: false)
</body>
```

The purpose of this section is including page-specific JavaScript files within the layout, so they appear after global script files. That way page-specific files can make use of the content of the global files.

The `IsSectionDefined` method can be used in the layout page to determine whether the content page has defined a specific section. This method can be useful, for example, if you want to display some default content if the calling page doesn't define the section.